Runbook — Confluent Platform + Flink on minikube

End-to-end operational guide for running `confluent-kafka-isotope` locally on **minikube**: cluster → Kafka/Confluent Platform → Flink → SQL reports → traffic → observe → teardown. Every step maps to a target in the Makefile, which is the source of truth.

> This is the **Confluent Platform + Flink (self-managed) on minikube** path. The Confluent Cloud for Apache Flink (CCAF) path is entirely different — Terraform-driven via `make cc-flink-reports-up`, no minikube — see root README §3.3.

Table of Contents

- 1.0 Prerequisites (once per machine)
- 2.0 Cluster + Confluent Platform
- 3.0 Flink
- 4.0 Port-forward Kafka
- 5.0 Deploy the 7 Flink reports (as a CMF Application)
- 6.0 Drive traffic (required to see report rows)
- 7.0 Observe
  - 7.1 [OPTIONAL] Showcase the two provenance collectors
    - 7.1.1 Fan-in (Merge) Provenance — Three things to point at
      - 7.1.1.1 The merged record carries a *fresh* trace, not a stolen one
      - 7.1.1.2 The merge trace ID is derived, not minted — the window is readable inside it
      - 7.1.1.3 Both reconciliations hold exactly
    - 7.1.2 State Provenance — Three things to point at
      - 7.1.2.1 The version chain, inline
      - 7.1.2.2 The whole topic is internally consistent
      - 7.1.2.3 No hops anywhere
    - 7.1.3 The punchline — one order, three registers
- 8.0 Teardown
- 9.0 Minimal path (no Flink)
- 10.0 Troubleshooting

1.0 Prerequisites (once per machine)

```
make install-prereqs     # VM driver (vfkit | kvm2/qemu), kubectl,
minikube, helm, gettext, gradle, openjdk17 — on Linux: sudo make install-
prereqs
make check-prereqs       # verify they're on PATH
```

Default minikube sizing: `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480` (MiB), `MINIKUBE_DISK=50g`. Override via env, `make` args, or `local.mk`. Sizing is fixed when the cluster is created, so run `make`

`minikube-delete` before changing it. The node architecture is auto-detected so the right `cp-flink` image (amd64/arm64) is selected.

minikube runs as a **VM with containerd** as the container runtime, so Docker isn't required. The VM driver is picked per OS and can be overridden with `MINIKUBE_DRIVER=`:

Host	Default driver	Alternative
macOS (Apple silicon or Intel)	`vfkit` (Apple Virtualization framework)	—
Linux x86_64	`kvm2` (KVM via libvirt)	`qemu`
Linux arm64	`qemu`	—

On Linux, both drivers need hardware virtualization (`/dev/kvm`; on a cloud VM that means nested virtualization). `sudo make install-prereqs` adds your user to the `kvm` group (plus `libvirt` for kvm2), so log out and back in afterwards. Run `make minikube-driver-check` to verify the driver setup. **Node can't pull images but the host can?** Set `MINIKUBE_HTTP_PROXY` to an HTTP proxy on the host in a git-ignored `local.mk` (see `local.mk.example`), and see KNOWN_ISSUES.md 2.0.

2.0 Cluster + Confluent Platform

```
make cp-up                # = check-prereqs → minikube-start → operator-
install → cp-deploy
make cp-watch             # watch pods come up (Ctrl+C to exit); or: make
cp-status
```

`cp-up` boots minikube, installs the CFK (Confluent for Kubernetes) operator, and deploys Kafka (KRaft) + Schema Registry + Connect + ksqlDB + REST Proxy + Control Center into the `confluent` namespace.

> `cp-up` deliberately does **not** bring up Flink — run §3.0 Flink separately.

3.0 Flink

```
make cp-flink-up          # cert-manager → operator → RustFS → CMF 2.5 → env
→ RBAC → app image
make cmf-status           # (~5 min the first time)
```

This installs cert-manager, the Confluent Flink Kubernetes Operator, **RustFS** (S3-compatible store for CMF artifacts), **CMF 2.5** (with artifact management + the writable environment catalog enabled), creates the `dev-local` Flink environment, applies the Flink RBAC, and builds the custom `cp-flink` 2.1 image (`isotope-cp-flink-sql:local`) that bakes in the Kafka + Avro SQL connectors and the S3 filesystem plugin.

> **The reports run as a CMF *Application*, not SQL statements.** CMF's SQL-statement runtime (`io.confluent.flink.FlinkCompiledPlanExecutor`) ships only in the `cp-flink-sql` image, which exists only at **Flink 1.19** — and two reports are `ProcessTableFunction`s, a **Flink**

> **2.x** feature. So all seven reports deploy as a single Flink 2.1 CMF **Application** (entry point `IsotopeReportsJob`), which runs the same `.fql` and surfaces in CMF / Control Center as a managed application. No raw session cluster is created — if you want one for ad-hoc SQL, deploy it separately with `make flink-deploy` (see §7.0 Observe).

> **CMF license.** The `cp-cmf` image ships with a date-locked trial license. Once it expires, the CMF pod `CrashLoopBackOff`s on startup (`LicenseInitiator` throws) and `make cp-flink-up` fails at the CMF readiness wait. To run past expiry, supply a Confluent license via a secret and point `CMF_LICENSE_SECRET` at it:
>
> ```
> kubectl create secret generic confluent-license-for-cmf -n confluent \
>   --from-file=license.txt=/path/to/license.txt   # key MUST be
> license.txt
> make cp-flink-up CMF_LICENSE_SECRET=confluent-license-for-cmf
> ```
>
> Export `CMF_LICENSE_SECRET` in your shell to make plain `make cp-flink-up` pick it up.

4.0 Port-forward Kafka

```
make kafka-pf-up          # localhost:30092 → Kafka, localhost:8081 → SR
```

Prereq for everything the host-run gradle app does — `App.java`'s defaults already point at `localhost:30092` / `localhost:8081`. Stop with `make kafka-pf-down`.

5.0 Deploy the 7 Flink reports (as a CMF Application)

```
make cp-flink-reports-up    # builds the app shadow JAR (:ptf:shadowJar) →
pre-creates
                            # 4 source + 7 sink topics → uploads the JAR as
a cmf://
                            # artifact (RustFS) → deploys the
FlinkApplication → waits RUNNING
```

All seven reports run in **one** Flink 2.1 CMF Application (`isotope-reports`): five pure Flink SQL plus two JAR-backed `ProcessTableFunction`s (`LatencyPercentilesPTF`, `StuckTracePTF`), executed together as a single `StatementSet` by `IsotopeReportsJob`. Sink topics use Apache Flink's `avro-confluent` format — SR-framed Avro, auto-registered on first write — so Control Center renders the rows natively. The application appears in CMF's applications API and Control Center's Flink tab. Drop everything (application + artifact + sink topics) with `make cp-flink-reports-down`.

Optional — fan-in provenance. Off by default. To also run the merge collector and its merge-edge sideband:

```
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
```

That adds two topics (`orders.flink_batched`, `isotope_merge_edge_markers`) and two more `INSERT`s to the same `StatementSet`. The merged records carry a fresh trace, and the edge topic records which parent traces fed each one — one row per contributing trace per window, weighted by `contributing_records`, a material fraction of the merge stage's write volume. See docs/flink-collector.md §2.4. Teardown removes those topics either way.

[OPTIONAL] State-level provenance (§3.6). A second, independent switch:

```
make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true

# both collectors — they answer different questions
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
ENABLE_STATE_PROVENANCE=true
```

That adds one topic (`isotope_state_provenance`) and one `INSERT` to the same `StatementSet`. Each traced order becomes an entity whose three stage records are three versions of it, chained by a `parents` array — `orders.placed` → `orders.enriched` → `orders.fulfilled`. Unlike the merge collector this needs no window: identity is content-addressed, so it works on the updating sources (unbounded `GROUP BY`, joins, dedup) where the windowed path has nothing to key on. CCAF runs the same statement with `make cc-flink-reports-up ENABLE_STATE_PROVENANCE=true` and derives the same version IDs. See docs/state-provenance.md §5.0. Teardown removes the topic either way.

Watch it fill. The topic is `avro-confluent`, and `kafka-0` ships no Avro-aware consumer — `kafka-avro-console-consumer` lives in the Schema Registry pod, so run it there:

```
kubectl exec -n confluent schemaregistry-0 -- kafka-avro-console-consumer \
    --bootstrap-server kafka.confluent.svc.cluster.local:9071 \
    --property schema.registry.url=http://localhost:8081 \
    --topic isotope_state_provenance --from-beginning --max-messages 5
```

Both optional collectors have a guided walkthrough — §7.1.

6.0 Drive traffic (required to see report rows)

All report jobs aggregate over `TUMBLE(event_time, INTERVAL '1' MINUTE)` windows, which only emit when the watermark advances past `window_end`. Spread records across **multiple** windows.

Start the three pipeline stages, each in its own terminal. They are long-running consumers that idle until traffic arrives:

```
./gradlew :app:run --args="enrich"        # orders.placed   →
orders.enriched
```

```
./gradlew :app:run --args="fulfill"       # orders.enriched → orders.fulfilled
./gradlew :app:run --args="ship"          # terminal consume orders.fulfilled
```

Then, in a fourth terminal, drive the traffic through them:

```
# 30 records, 5s apart ≈ 2.5 min of event-time → spans 3+ windows
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep 5; done
```

Event time only advances when records arrive, so waiting alone won't close the newest window or alert on the newest traces: keep some traffic flowing until the watermark passes them. A topic that goes quiet stops holding the watermark back after 30s (`table.exec.source.idle-timeout`), so one stopped stage doesn't freeze the reports. `stuck_trace_alerts_1m` only fires for a trace that goes ≥60s of event time without a fresh hop and has not reached `orders.fulfilled`; to exercise it, **stop the enrich/fulfill stages**, send one record to `orders.placed`, and keep sending unrelated records so the watermark crosses `event_time + 60s`.

7.0 Observe

```
make c3-open              # Control Center — report topics render natively (Avro+SR)
make flink-ui             # Flink job graph / watermarks for the reports (localhost:8082)
make metrics-up           # optional: Prometheus + Grafana for the stateless meters
```

The metrics showcase has its own runbook — k8s/monitoring/README.md (and the meter/PromQL reference in docs/metrics.md). Stop the background port-forwards with `make c3-stop` / `make flink-ui-stop` / `make metrics-down`.

Ad-hoc Flink SQL — needs a session cluster (optional). The reports run as a CMF *Application* (§5.0): one compiled job under `execution.target=kubernetes-application`, which **cannot accept ad-hoc SQL**. Interactive querying needs a separate **session cluster**, and neither `cp-flink-up` nor `cp-flink-reports-up` creates one. Deploy it first:

```
make flink-deploy         # one-time: deploy the 'flink-basic' session cluster (~40s)
make flink-sql            # interactive SQL Client on it, report DDL preloaded
```

`flink-deploy` applies k8s/base/flink-cluster-deployment.yaml — a `cp-flink` 2.1 session cluster with 8 task slots, whose init containers fetch the Kafka and Avro-SR connector JARs. `flink-sql` then installs

the PTF JAR into the JobManager's `/opt/flink/lib` and preloads the DDL from scripts/flink/sql/cp/, so the session opens ready to query:

```
Flink SQL> SHOW TABLES;          -- 4 source tables + 7 report sinks + 2
merge tables + views
Flink SQL> SHOW USER FUNCTIONS;  -- latency_percentiles, stuck_trace_ptf
```

Skip `flink-deploy` and `make flink-sql` stops with a message telling you to run it — it will not fall back to the reports Application pod. `make flink-ui` opens the reports Application by default; point it at this session cluster with `make flink-ui FLINK_UI_TARGET=flink-basic` to watch ad-hoc queries.

> **The SQL Client catalog is its own.** The preloaded DDL lives in that session only, and the running reports Application registers its tables inside the job's own `TableEnvironment` — no shared catalog exists, so `SHOW TABLES` never reflects what the Application is running. Only DDL is preloaded; the report `INSERT INTO` files (`10_`…`70_`) are deliberately excluded, because the SQL Client rejects DML in an init file. Run them by hand only if you mean to — the Application is already writing to those same sink topics, so you would get duplicate rows, not an error.

7.1 [OPTIONAL] Showcase the two provenance collectors

Both collectors from §5.0 have to be on, and §6.0 traffic has to have run — including the `enrich` and `fulfill` stages, or every state entity has exactly one version and there is no chain to show:

```
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
ENABLE_STATE_PROVENANCE=true
```

> **Note about rendering the topics in Control Center**: `make c3-open`, then Topics → any of `orders.flink_batched`, `isotope_merge_edge_markers`, `isotope_state_provenance` — all three are SR-framed Avro and render natively. If C3 will not come up, it is almost certainly the startup race in KNOWN_ISSUES.md §1.0.

Showcasing the **two** provenance collectors takes both consumers, and they live in **different pods** — set them up before you present:

Topic	Format	Consumer	Pod
`orders.*` (sources, `flink_enriched`)	`raw`	`kafka-console-consumer`	`kafka-0`
`orders.flink_batched`, `isotope_merge_edge_markers`, `isotope_state_provenance`	`avro-confluent`	`kafka-avro-console-consumer`	`schemaregistry-0`

Both write a deprecation warning and a `--timeout-ms` shutdown trace to **stdout**, so both helpers keep only the data lines:

```
# Avro+SR sinks — schemaregistry-0 is the only pod with the Avro consumer.
avro() { kubectl exec -n confluent schemaregistry-0 -- kafka-avro-console-
consumer \
    --bootstrap-server kafka.confluent.svc.cluster.local:9071 \
    --property schema.registry.url=http://localhost:8081 \
    --from-beginning --timeout-ms 20000 "$@" 2>/dev/null | grep -E '^(\{|x-
isotope|NO_HEADERS)'; }

# raw-format order topics — kafka-0 is the only pod with the plain
consumer.
# grep -a matters: the values are binary, so without it grep reports
# "binary file matches" instead of printing the header line.
plain() { kubectl exec -n confluent kafka-0 -- kafka-console-consumer \
    --bootstrap-server localhost:9071 --from-beginning --timeout-ms 20000 \
    --property print.headers=true \
    --property
headers.deserializer=org.apache.kafka.common.serialization.StringDeserializ
er \
    "$@" 2>/dev/null | grep -a '^x-isotope'; }
```

7.1.1 Fan-in (Merge) Provenance — Three things to point at

7.1.1.1 The merged record carries a *fresh* trace, not a stolen one

```
avro --topic orders.flink_batched --max-messages 1 --property
print.headers=true \
     --property
headers.deserializer=org.apache.kafka.common.serialization.StringDeserializ
er \
| tr ',' '\n' | grep -aE 'trace-id|hop-count|origin-service|origin-ts'
```

```
x-isotope-trace-id:01a073036f007244ae76b286abd25c7f
x-isotope-origin-service:flink-batch
x-isotope-hop-count:1
x-isotope-origin-ts:1788636000000
```

The payload it is stamped on is the batch summary itself:

```
avro --topic orders.flink_batched --max-messages 1
```

```
{"pipeline":{"string":"orders"},"window_start":
{"long":1788635940000},"window_end":{"long":1788636000000},"event_count":
```

```
{"long":2},"distinct_traces":{"long":2}}
```

Now put the always-on 1:1 collector beside it:

```
plain --topic orders.flink_enriched --max-messages 1 \
| tr ',' '\n' | grep -aE 'trace-id|hop-count|origin-service|this-service'
```

```
x-isotope-trace-id:01a07302885b726faa1184fc79d6f984
x-isotope-origin-service:order-intake-service
x-isotope-this-service:flink-enrich
x-isotope-hop-count:2
```

That single digit is the whole argument. Same UDF family, same header shape, two different answers: the 1:1 forward **continued** an identity that `order-intake-service` minted (`hop_count = 2`, origin unchanged), and the merge **started** one (`hop_count = 1`, `origin_service = flink-batch`). Nobody told either statement which it was — the count says it. That is the flink-collector.md §3.1 rule made visible: a trace is truthful only while every step has exactly one parent, so the merge does not get to claim one.

7.1.1.2 The merge trace ID is *derived*, not minted — the window is readable inside it

It is already visible in the output above: the trace ID opens with `01a073036f00`, and `0x01a073036f00` is `1788636000000` — the `window_end`. The first 12 hex characters are the UUIDv7 timestamp field, and the merge collector fills it from the window rather than from a clock:

```
01a073036f00 → 1788636000000 = window_end
01a073045960 → 1788636060000 = window_end
01a0730543c0 → 1788636120000 = window_end
```

This is why `80_merge_collector.fql` and `81_merge_edge_markers.fql` can agree on an ID without ever exchanging one, and why a replayed window reproduces it byte for byte.

7.1.1.3 Both reconciliations hold exactly

Join the edges to the merged records on `merge_trace_id` and check the two identities flink-collector.md §2.4 asserts:

```
avro --topic orders.flink_batched --property print.headers=true \
     --property
headers.deserializer=org.apache.kafka.common.serialization.StringDeserializ
er \
| sed -E 's/.*x-isotope-trace-id:([0-9a-f]{32}).*\t(\{.*\})$/\1 \2/' >
/tmp/batched.txt
```

```
avro --topic isotope_merge_edge_markers > /tmp/edges.json

while read -r tid json; do
    ec=$(jq -r '.event_count.long'     <<<"$json")
    dt=$(jq -r '.distinct_traces.long' <<<"$json")
    edges=$(grep -c "\"$tid\"" /tmp/edges.json)
    recs=$(grep "\"$tid\"" /tmp/edges.json | jq -s
'map(.contributing_records.long)|add')
    printf '%s  edges=%-3s traces=%-3s  records=%-3s events=%-3s  %s\n' \
        "$tid" "$edges" "$dt" "$recs" "$ec" \
        "$([ "$edges" = "$dt" ] && [ "$recs" = "$ec" ] && echo OK || echo
MISMATCH)"
done < /tmp/batched.txt
```

```
01a073036f007244ae76b286abd25c7f  edges=2   traces=2    records=2
events=2    OK
01a073045960714db1a35a4c0b28f701  edges=9   traces=9    records=9
events=9    OK
01a0730543c07df7b8b84e482fbd9c07  edges=10  traces=10   records=10
events=10   OK
```

`COUNT(*) == distinct_traces` and `SUM(contributing_records) == event_count`. Both hold because the two statements sit behind the same window operator — a late record is dropped by both or by neither.

> **What this demo cannot show.** Every trace touches `orders.placed` exactly once, so `contributing_records` is always `1` and `event_count` always equals `distinct_traces`. The weighted-edge column is exercised in shape, not in value; a source where one trace contributes several records to a window is what makes the two reconciliations differ.

> **Which merged records did a stuck trace feed?** With merge provenance on and a stuck trace exercised, paste the two queries from `82_merge_impact_query.fql` into `make flink-sql`. Both run until you cancel them, so stop them once you have your answer. The session preloads the merge table definitions, but those topics only have data if the reports are running with `ENABLE_MERGE_PROVENANCE=true`.

7.1.2 State Provenance — Three things to point at

7.1.2.1 The version chain, inline

One entity, three versions, each naming its predecessor — no join, no second topic:

```
avro --topic isotope_state_provenance > /tmp/state.json
E=$(head -1 /tmp/state.json | jq -r .entity_key.string)
jq -r --arg e "$E" 'select(.entity_key.string==$e)
    | "\(.source_name.string)\t\(.version_id.string)\tparents=\
```

```
(.parents.array|map(.string)|join(","))"' /tmp/state.json \
| column -t -s$'\t'
```

```
orders.placed     01a0730288817a44bb84484659ccfa6f  parents=
orders.enriched   01a07302bdde75c784f31084f187f5ea
parents=01a0730288817a44bb84484659ccfa6f
orders.fulfilled  01a07303085d7a6c8306afcc99abfb66
parents=01a07302bdde75c784f31084f187f5ea
```

The `DemoEvent` payload is forwarded verbatim across all three hops, so the **bytes never change** — and these are still three distinct versions, because `source_name` is part of the preimage (state-provenance.md §2.1).

7.1.2.2 The whole topic is internally consistent

Every parent resolves to a version of the same entity, one root per entity, nothing overflowed:

```
jq -s 'INDEX(.version_id.string) as $v
    | {versions: length,
       entities: (map(.entity_key.string)|unique|length),
       roots:    (map(select(.parents.array|length==0))|length),
       links:    (map(select(.parents.array|length>0))|length),
       every_parent_same_entity:
           (map(select(.parents.array|length>0)
                | .entity_key.string ==
$v[.parents.array[0].string].entity_key.string) | all),
       overflow: (map(select(.parent_overflow.int>0))|length)}'
/tmp/state.json
```

```
{
  "versions": 93,
  "entities": 31,
  "roots": 31,
  "links": 62,
  "every_parent_same_entity": true,
  "overflow": 0
}
```

31 traced orders × 3 stages. Point out that the parent set is a **column of the record it describes**, so `versions` and `links` cannot drift the way `event_count` and the merge edge list could — that drift is not expressible here.

7.1.2.3 No hops anywhere

Ask the same consumer to print headers, and the records have none at all:

```
avro --topic isotope_state_provenance --max-messages 3 --property
print.headers=true \
     --property
headers.deserializer=org.apache.kafka.common.serialization.StringDeserializ
er \
| cut -c1-60
```

```
NO_HEADERS  {"version_id":{"string":"01a0730288817a44bb844846
NO_HEADERS  {"version_id":{"string":"01a07302bdde75c784f31084
NO_HEADERS  {"version_id":{"string":"01a07303085d7a6c8306afcc
```

Not "no hop count" — **no isotope at all**. A row being revised is not a record taking a hop, so this collector declines the question rather than answering it wrong. Message lineage stays with the isotope on the `orders.*` topics; state lineage is a parallel record keyed by version. The two coexist without either lying.

7.1.3 The punchline — one order, three registers

The strongest single moment of the demo. `entity_key` in the state topic **is** the isotope trace ID, which **is** `contributing_trace_id` in the merge edges, so the same order is addressable in all three lineage models at once.

This continues from the blocks above — it reuses `$E` from the state walkthrough and `/tmp/edges.json` from fan-in beat 3, so run those two first:

```
echo "trace $E"
plain --topic orders.flink_enriched --max-messages 1 | grep -ao "x-isotope-
hop-count:[0-9]*"
grep "\"$E\"" /tmp/edges.json \
| jq -r '"parent of merge_trace_id=\(.merge_trace_id.string) window=[\
(.window_start.long),\(.window_end.long)) records=\
(.contributing_records.long)"'
grep "\"$E\"" /tmp/state.json | jq -r '"version \(.source_name.string) = \
(.version_id.string)"'
```

```
trace 01a07302885b726faa1184fc79d6f984
x-isotope-hop-count:2
parent of merge_trace_id=01a073036f007244ae76b286abd25c7f window=
[1788635940000,1788636000000) records=1
version orders.placed = 01a0730288817a44bb84484659ccfa6f
version orders.enriched = 01a07302bdde75c784f31084f187f5ea
version orders.fulfilled = 01a07303085d7a6c8306afcc99abfb66
```

One order, read three ways: **as a message** it is an itinerary carried in-band; **as a fan-in parent** it is an edge recorded out-of-band, because the merge could not carry it; **as an entity** it is a version chain that never mentions hops. Three provenance models, one identity, and **no change to the isotope wire format** for any of them.

8.0 Teardown

Pick the depth:

```
make cp-flink-reports-down  # drop reports / views / functions only
make flink-delete           # drop just the 'flink-basic' ad-hoc SQL
session cluster
make metrics-delete         # remove the Prometheus/Grafana showcase (pods
+ namespace)
make cp-flink-down          # Flink cluster + CMF + operator + cert-manager
(includes flink-delete)
make cp-down                # CP + operator (keeps minikube running)
make cp-teardown            # everything + stop minikube
make nuke                   # cp-teardown + minikube-delete + uninstall-
prereqs (factory reset)
```

9.0 Minimal path (no Flink)

To just watch a trace propagate without the Flink SQL reports:

```
make cp-up
make kafka-pf-up
./gradlew :app:run --args="place hello"   # then enrich / fulfill / ship
```

Flink (§3.0 Flink) and the reports (§5.0 Deploy the 7 Flink reports (as a CMF Application)) are only needed for the SQL report topics.

10.0 Troubleshooting

- **Pods stuck `Pending`.** minikube is under-resourced — raise `MINIKUBE_CPUS` / `MINIKUBE_MEM` and `make minikube-delete && make cp-up`.
- **Flink job submission fails (`services` forbidden).** The supplemental RBAC didn't apply — `make flink-rbac`.
- **No report rows.** Almost always the watermark — see §6.0 Drive traffic (required to see report rows); traffic must span multiple 1-minute windows and you must wait ~90s after the last record.
- **App can't reach Kafka.** `make kafka-pf-up` isn't running, or the forward died — re-run it and confirm `localhost:30092` / `localhost:8081` are live.
- **`make flink-sql` says no session cluster / `SHOW TABLES;` returns an empty set.** Ad-hoc SQL needs the `flink-basic` session cluster — run `make flink-deploy` first, see §7.0 Observe. Confirm with `kubectl get flinkdeployment -n confluent`: `isotope-reports` alone

means only the reports Application is up. Note that the report tables of the running Application are never visible to the SQL Client regardless — the catalogs are separate.

- **`controlcenter-0` sits at 2/3 and `make c3-open` gets a connection refused.** C3 lost the startup race against Kafka's DNS and cannot recover on its own — cause and workaround in KNOWN_ISSUES.md.
- **`make minikube-driver-check` fails.** It names the missing piece: `vfkit` (macOS), or on Linux `/dev/kvm` access, libvirt (`kvm2`), or `qemu-system-*` / `qemu-img` (`qemu`). Fix it with `make install-prereqs` (`sudo` on Linux) and log out and back in after any group change.
- **Pods stuck in `ErrImagePull` / `ImagePullBackOff` with `TLS handshake timeout`.** The minikube VM can't reach Docker Hub directly, usually because of a network filter or VPN on the host. Work around it with `MINIKUBE_HTTP_PROXY` — cause and workaround in KNOWN_ISSUES.md.
- **Control Center's Flink tab is blank.** CMF proxy connectivity — `make cmf-proxy-inject` (and `make cmf-proxy-logs` to debug).
- **`make cp-flink-up` times out waiting for the CMF pod / CMF `CrashLoopBackOff`.** Check `kubectl logs -n confluent -l app.kubernetes.io/name=confluent-manager-for-apache-flink`. If you see `Trial license ... expired` / `LicenseInitiator ... Constructor threw exception`, the image's embedded trial license has expired — supply your own via `CMF_LICENSE_SECRET` — see the CMF license note in §3.0 Flink. Wiping the CMF `PersistentVolumeClaim` does **not** reset it; the expiry is baked into the image.